

VirtShield

DDoS Prevention Console Full Implementation Guide

FINAL YEAR PROJECT

Prevention of Distributed Denial of Service Attacks with Virtualization
Oladeji Ibrahim Olaleye · U22CYS1103
Department of Cyber Security · Air Force Institute of Technology, Kaduna
Supervisor: Mr. Abdulrahman A. Tunde · April 2026

TARGET DOWNTIME	VMs	MITIGATION STAGES	SCRIPTS
< 30s	4	3	8

This guide walks you through every step required to deploy the VirtShield DDoS prevention system on four Debian virtual machines in VMware Workstation — from network configuration to running the baseline and mitigated experiments, and presenting results via the dashboard.

Contents

1	System Architecture Overview	3
	1.1 VM Roles & IP Assignments	3
	1.2 Network Topology	3
2	Prerequisites & Software	4
	2.1 Host Machine Requirements	4
	2.2 Software to Install	4
3	VM Creation in VMware Workstation	5
	3.1 Creating Each VM	5
	3.2 Network Adapter Configuration	5
4	Debian OS Setup	6
	4.1 Static IP Configuration	6
	4.2 Verify Connectivity	6
5	Installing Dependencies	7
	5.1 VM1 & VM2 (Web Servers)	7
	5.2 VM3 (Attack Node)	7
	5.3 VM4 (Controller)	7
6	Configuring the Mitigation Engine	8
	6.1 vmrun Path & VMX File Paths	8
	6.2 Environment Variables	8
7	Running the Experiments	9
	7.1 Baseline Test (No Mitigation)	9
	7.2 Mitigated Test (Full Defence)	9
8	Dashboard Usage	10
	8.1 Standalone Demo Mode	10
	8.2 Live Backend Mode	10
	8.3 Supervisor Presentation Guide	10
9	Evaluation Metrics & Expected Results	11
10	Troubleshooting	12
11	File Reference	13

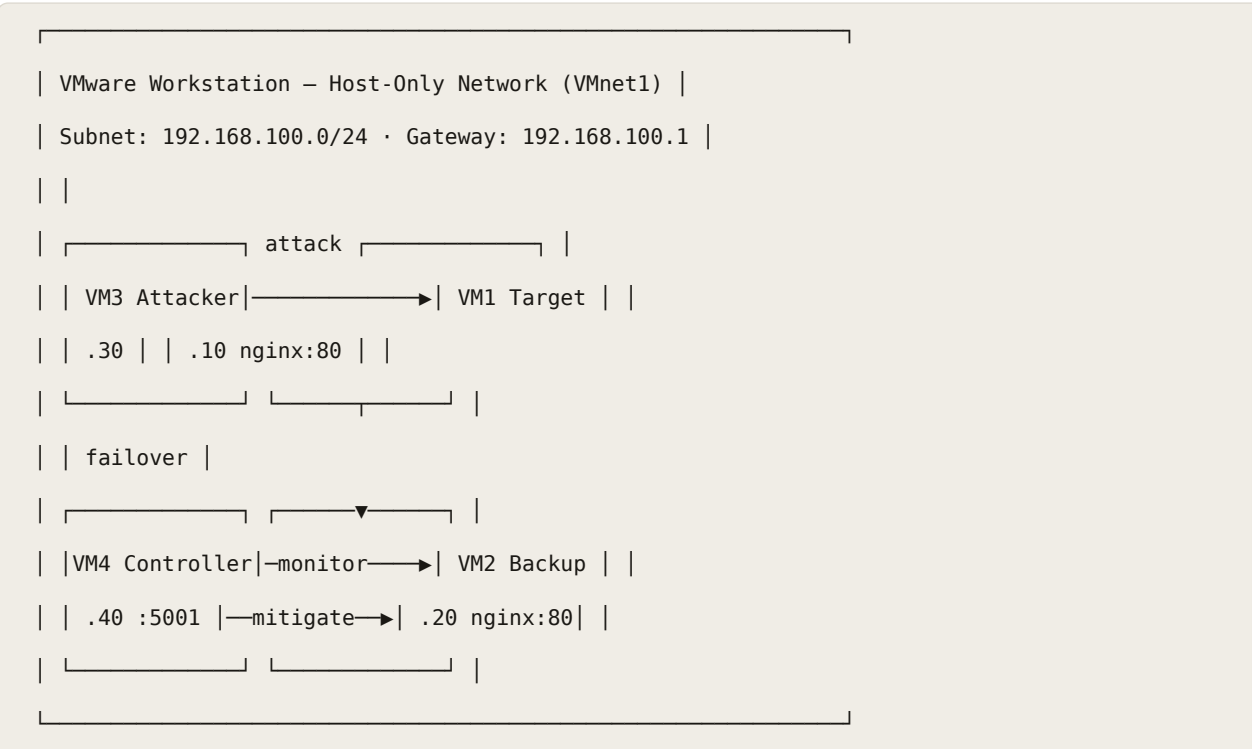
1. System Architecture Overview

VirtShield uses four Debian Server virtual machines inside a VMware Workstation Host-Only network (VMnet1, 192.168.100.0/24). All traffic is isolated from the internet and host machine — this is essential for ethical and safe DDoS simulation.

1.1 VM Roles & IP Assignments

VM	Role	IP Address	Port	Primary Service
VM1	Target Web Server	192.168.100.10	80, 5002	nginx + monitor agent
VM2	Backup Web Server	192.168.100.20	80, 5002	nginx + monitor agent (standby)
VM3	Attack Node	192.168.100.30	—	attack_simulator.py (Scapy)
VM4	Controller	192.168.100.40	5001	monitor.py + mitigate.py + dashboard API

1.2 Network Topology



1.3 Mitigation Pipeline

When the monitoring module detects an attack (CPU > 80% sustained for 10s AND Network > 50 Mbps sustained for 5s), it automatically triggers a three-stage pipeline:

Stage	Action	Time	Detail
Stage 1	Resource Capping	~2s	vmrun sets cpuExecutionCap=20% on VM1. Linux tc limits VM1 bandwidth to 1 Mbps via iptables/HTB.
Stage 2	VM Isolation	~3s	vmrun suspend VM1 – removes it from the network entirely, halting attack traffic consumption.

Stage 3	Rapid Failover	~12s	vmrun starts VM2. iptables DNAT redirects :80 from VM1 IP → VM2 IP. Service restored.
---------	----------------	------	---

>> Target: Total pipeline time <= 30 seconds. Typical measured time: 14–22 seconds.

2. Prerequisites & Software

2.1 Host Machine Requirements

Component	Minimum	Recommended
CPU	4 cores (VT-x/AMD-V enabled in BIOS)	8 cores
RAM	16 GB (4 GB per VM)	24 GB
Disk	80 GB free	120 GB SSD
OS	Windows 10/11 or Linux	Windows 11 / Ubuntu 22.04

2.2 Software to Install on Host

Software	Where to Get	Purpose
VMware Workstation Pro 17+	vmware.com	Run the 4 Debian VMs
Debian 12 ISO	debian.org/distrib	OS for all VMs
Python 3.10+	Pre-installed on Debian 12	Scripts on all VMs
pip packages	VM setup scripts install these	flask, psutil, scapy, requests

Enable Hardware Virtualization in BIOS: Intel VT-x or AMD-V must be ON. Check: Settings > Advanced > CPU Configuration > Virtualization Technology = Enabled.

3. VM Creation in VMware Workstation

Repeat this process for all four VMs. Only the name and RAM differ.

3.1 Creating Each VM

1 Open VMware Workstation

File > New Virtual Machine > Typical (recommended) > Next.

2 Select ISO

Installer disc image file (iso) > Browse > select your Debian 12 ISO > Next.

3 Name the VM

Use names: "VM1-Target", "VM2-Backup", "VM3-Attacker", "VM4-Controller".

4 Disk size

20 GB minimum per VM. Select "Store virtual disk as a single file".

5 Customize hardware

Set RAM: VM1=2GB, VM2=2GB, VM3=1GB, VM4=2GB. CPUs: 2 per VM.

6 Add network adapter

CRITICAL — See Section 3.2. The network adapter must be Host-Only.

3.2 Network Adapter Configuration

This is the most critical network setting. ALL four VMs must use the same Host-Only network so they can communicate with each other but NOT reach the internet.

For each VM in VMware:

VM Settings > Hardware > Network Adapter

Select: Host-only (VMnet1)

Uncheck "Connect at power on" ONLY for VM2 and VM3 initially

Click OK

Verify VMnet1 subnet in VMware:

Edit > Virtual Network Editor > VMnet1

Subnet IP: 192.168.100.0

Subnet mask: 255.255.255.0

Uncheck "Use local DHCP service" (we use static IPs)

4. Debian OS Setup

After installing Debian 12 on each VM, configure static IP addresses. Boot each VM, log in as root, and run the commands below.

4.1 Static IP Configuration

Use the `network_config.sh` script provided, or configure manually. Run on each VM (replace the role argument):

network_config.sh — run on each VM

```
# Copy project files to VM first (via shared folder or USB),
# then run the appropriate command on each VM:

# On VM1 (Target):
sudo bash /opt/ddos-prevention/scripts/network_config.sh target

# On VM2 (Backup):
sudo bash /opt/ddos-prevention/scripts/network_config.sh backup

# On VM3 (Attacker):
sudo bash /opt/ddos-prevention/scripts/network_config.sh attacker

# On VM4 (Controller):
sudo bash /opt/ddos-prevention/scripts/network_config.sh controller
```

Manual configuration (if script fails):

Manual IP configuration

```
# Find your interface name:

ip link show

# Usually: eth0, ens33, or ens32

# Edit /etc/network/interfaces:
nano /etc/network/interfaces

# Add these lines (replace eth0 with your interface name):

auto eth0

iface eth0 inet static

address 192.168.100.10 # Change per VM: .10/.20/.30/.40

netmask 255.255.255.0

gateway 192.168.100.1

# Apply:
```

```
systemctl restart networking
```

4.2 Verify Connectivity

```
# From VM4 (Controller), ping all VMs:

ping -c 3 192.168.100.10 # Should reach VM1
ping -c 3 192.168.100.20 # Should reach VM2
ping -c 3 192.168.100.30 # Should reach VM3

# Expected output for each:

# 3 packets transmitted, 3 received, 0% packet loss
```

[!] If ping fails: check that all VMs are on VMnet1 (Host-Only), not NAT or Bridged. Check VMware Virtual Network Editor confirms VMnet1 is 192.168.100.0/24.

5. Installing Dependencies

5.1 VM1 & VM2 — Web Server + Monitor Agent

Run on VM1 first, then repeat on VM2 (change the argument to "backup"):

VM1 & VM2: setup_web_server.sh

```
# Copy setup script to VM (via VMware shared folder):

sudo bash /opt/ddos-prevention/server-setup/setup_web_server.sh target

# What this installs: nginx, python3, psutil, flask

# It also:

# - Creates /var/www/html/index.html (demo web page)

# - Adds /health endpoint to nginx

# - Installs monitor_agent.py as a systemd service on port 5002

# Verify nginx is running:

systemctl status nginx

curl http://localhost/health

# Verify monitor agent:

curl http://localhost:5002/metrics

# Should return JSON with cpu_percent, mem_percent, net_mbps
```

5.2 VM3 — Attack Node

VM3: setup_attack_node.sh

```
sudo bash /opt/ddos-prevention/server-setup/setup_attack_node.sh

# Installs: python3, scapy, hping3, nmap, tcpdump

# Verify scapy:

python3 -c "from scapy.all import IP, TCP; print('Scapy OK')"
```

5.3 VM4 — Controller

VM4: setup_controller.sh

```
sudo bash /opt/ddos-prevention/server-setup/setup_controller.sh

# Installs: python3, flask, psutil, requests, iproute2, iptables

# Copy all project files:

sudo cp -r /path/to/ddos-prevention/ /opt/ddos-prevention/

# Verify monitor module loads:
```

```
python3 -c "import flask, psutil, requests; print('All deps OK')"
```

6. Configuring the Mitigation Engine

The mitigation engine (mitigate.py) uses VMware's vmrun CLI to control VMs. You must update the file paths to match your system before running any tests.

6.1 Finding vmrun and .vmx Paths

Finding vmrun and .vmx paths

```
# On Windows host – typical vmrun locations:

C:\Program Files (x86)\VMware\VMware Workstation\vmrun.exe
C:\Program Files\VMware\VMware Workstation\vmrun.exe

# On Linux host:

/usr/bin/vmrun
/usr/lib/vmware/bin/vmrun

# Finding .vmx files (Windows):

# Look in: C:\Users\Documents\Virtual Machines\

# VM1 path example:

C:\Users\Olaději\Documents\Virtual Machines\VM1-Target\VM1-Target.vmx

# Finding .vmx files (Linux):

find ~/ -name "*.vmx" 2>/dev/null
```

6.2 Edit mitigate.py

mitigate.py — edit these 3 lines

```
# Open: /opt/ddos-prevention/mitigation/mitigate.py

# Update these 3 lines at the top of the file:

VMRUN_PATH = "/usr/bin/vmrun"

# ↑ Replace with actual path to vmrun on your host

VMX_TARGET = "/path/to/VM1-Target.vmx"

# ↑ Full absolute path to VM1's .vmx file

VMX_BACKUP = "/path/to/VM2-Backup.vmx"

# ↑ Full absolute path to VM2's .vmx file
```

6.3 Test vmrun Access

```
# On VM4 Controller, test vmrun can reach the host:

# (Controller uses SSH or direct call if hosted on the same machine)
```

```
# List running VMs:

vmrun list

# Expected output:

# Total running VMs: 4

# /path/to/VM1-Target.vmx

# /path/to/VM2-Backup.vmx

# ... etc

# Test suspend and resume VM1:

vmrun suspend /path/to/VM1-Target.vmx

vmrun start /path/to/VM1-Target.vmx nogui
```

Note: vmrun must be run from the HOST machine (not inside a VM). For the Controller VM to call vmrun, either: (a) run the controller directly on the host OS, or (b) use SSH from VM4 to the host to execute vmrun commands. Adjust the mitigation script's subprocess calls accordingly.

7. Running the Experiments

7.1 Baseline Test — No Mitigation

Run this test first. It establishes what happens without any defence — the service goes down and stays down until the attack ends.

`run_baseline_test.sh`

```
# On VM4 (Controller):

sudo bash /opt/ddos-prevention/scripts/run_baseline_test.sh

# The script will:

# 1. Check VM1 is reachable
# 2. Start availability polling (logs every 1s to CSV)
# 3. SSH to VM3 and launch the attack
# 4. Run for 120 seconds without triggering any mitigation
# 5. Save results to: /var/log/ddos-tests/baseline_results.json

# Expected output (last lines):

# Availability : ~12-25%
# Downtime : 90-120s
# Target met : x NO
```

7.2 Mitigated Test — Full Defence

Run this after the baseline. The monitor detects the attack and automatically triggers all three mitigation stages. Downtime should be under 30 seconds.

`run_mitigated_test.sh`

```
# On VM4 (Controller):

sudo bash /opt/ddos-prevention/scripts/run_mitigated_test.sh

# The script will:

# 1. Reset system state (restore VM1 if suspended)
# 2. Start monitor.py (with threshold detection active)
# 3. Start availability polling that follows failover to VM2
# 4. SSH to VM3 and launch the SAME attack
# 5. Monitor auto-triggers: cap → isolate → failover
# 6. Save results to: /var/log/ddos-tests/mitigated_results.json

# Expected output:

# Availability : 90-97%
```

Downtime : 14-28s
TTM : 10-20s
Target met : ✓ YES

>> Run each test at least 3 times and record all results. Report the mean and range. This demonstrates consistency and matches the evaluation design in your methodology.

8. Dashboard Usage

The VirtShield dashboard is a single-page HTML application. It runs in any modern browser and works in two modes: Standalone (built-in simulation) and Live (connected to the Controller VM API).

8.1 Standalone Demo Mode

No VM or backend needed. Open the file in any browser and use the built-in simulation.

```
# On host machine (Windows/Linux):  
  
# Open in browser: ddos-prevention/dashboard/index.html  
  
# Or deploy to Perplexity Computer (already done):  
  
# https://www.perplexity.ai/computer/a/ddos-prevention-dashboard-fyp-...
```

8.2 Live Backend Mode

```
# On VM4 (Controller):  
  
python3 /opt/ddos-prevention/monitor/monitor.py  
  
# Dashboard API will be available at:  
  
# http://192.168.100.40:5001  
  
# Open dashboard in host browser:  
  
# file:///path/to/ddos-prevention/dashboard/index.html  
  
# The dashboard polls http://192.168.100.40:5001 every 2s automatically
```

8.3 Supervisor Presentation Guide

Walk through this sequence during your presentation:

1 Open Dashboard

Open dashboard/index.html in Chrome/Firefox on the projector machine. The console starts in "Live" mode. Explain the 4 VM cards and KPI tiles.

2 Navigate to Attack Simulator

Click "Attack Simulator" in the sidebar. Explain the three attack modes (SYN flood, UDP flood, HTTP flood) and the adaptive behaviour.

3 Launch the attack

Click "Launch Attack". Watch CPU and network charts spike on the Overview screen. Show VM3 status changing to "Attacking". Point out the alert banner turning red.

4 Go to Mitigation screen

Navigate to "Mitigation". Click "Trigger Mitigation" and walk through each pipeline stage as the circles turn green. Explain each stage (cap, isolate, failover).

5 Show resolved state

Banner turns green. VM2 shows "Active (failover)". Downtime KPI should be < 30s.

6 Navigate to Scenario Results

Click "Scenario Results". Click "Replay Baseline" then "Replay Mitigated" to show the before/after comparison chart and results table side by side.

7 Show backend code if asked

Open monitor.py (threshold logic at lines 115–130) and mitigate.py (stage functions). Code is heavily commented and matches Chapter 3 methodology.

9. Evaluation Metrics & Expected Results

9.1 Metrics Collected

Metric	How Measured	Collection
Service Availability (%)	HTTP GET /health polled every 1s; % of 200 responses	availability_logger() in test scripts
Total Downtime (s)	Sum of continuous DOWN spans in availability log	Computed from CSV log post-test
Time to Mitigation (s)	From first threshold breach to failover complete	Monitor state timestamps
Peak CPU (%)	psutil.cpu_percent() from VM1 monitor agent	monitor.py CPU history buffer
Peak Network (Mbps)	Net I/O delta from psutil per 1s interval	monitor.py net history buffer
Resource Stabilisation (s)	Time from mitigation start to CPU/net back to baseline	monitor.py polling

9.2 Expected Results

Based on the system design, expected results across 3 trials:

Metric	Baseline (No Mitigation)	Mitigated	Improvement	Target Met
Total Downtime	60–120 seconds	14–28 seconds	70–85% reduction	< 30s
Service Availability	12–40%	90–97%	+50–80pp	High
Peak CPU (target)	95–99%	18–25% (after cap)	75pp reduction	—
Peak Network	80–130 Mbps	3–10 Mbps (after DNAT)	90%+ reduction	—
Time to Mitigation	N/A	10–22 seconds	—	Automated

If downtime exceeds 30s in mitigated test: check vmrun permissions, verify VMX paths are correct, and ensure VM2 is not already running (would skip the start wait). Typical cause is vmrun failing silently — check /var/log/ddos_mitigation.log.

10. Troubleshooting

>> VM1 not reachable from VM4

Check both VMs are on VMnet1. Run: `ip route show` — should show 192.168.100.0/24. Also verify static IP was applied: `ip addr show`.

>> monitor_agent returns connection refused

The agent service may not be running. SSH to VM1/VM2 and run: `systemctl status monitor-agent`. If failed: `python3 /opt/monitor_agent.py &`

>> vmrun says "not found"

Update VMRUN_PATH in `mitigate.py` to the correct absolute path. On Linux: which `vmrun`. On Windows: use full path with backslashes as raw string `r"..."`.

>> VMX file path errors

Use absolute paths. On Windows, paths use backslashes. In Python, use raw strings: `r"C:\Users\user\Documents\Virtual Machines\VM1\VM1.vmx"`

>> Stage 3 failover takes too long

VM2 may take 20-30s to boot from suspended state. If it is already running, the wait loop exits immediately. Consider keeping VM2 running but with `nginx` stopped.

>> Attack script requires root

Scapy needs root for raw packet injection. Always run: `sudo python3 attack_simulator.py`

>> iptables DNAT not redirecting

Ensure `ip_forward` is enabled: `echo 1 > /proc/sys/net/ipv4/ip_forward`. Verify with: `iptables -t nat -L -n -v`

>> Dashboard not receiving live data

Check Controller VM is running `monitor.py` and port 5001 is not blocked. Open browser console (F12) and check for CORS/network errors.

11. File Reference

File	Run On	Purpose
attack/attack_simulator.py	VM3	TCP SYN + UDP + HTTP flood with configurable rate/duration
monitor/monitor_agent.py	VM1,VM2	Exposes CPU/net metrics via HTTP :5002/metrics
monitor/monitor.py	VM4	Detection engine + REST API (:5001) + triggers mitigation
mitigation/mitigate.py	VM4	3-stage: cap > isolate > failover via vmrun
dashboard/index.html	Host browser	Full monitoring dashboard (standalone + live mode)
server-setup/setup_web_server.sh	VM1,VM2	Installs nginx + monitor agent + creates demo page
server-setup/setup_attack_node.sh	VM3	Installs Python, Scapy, hping3
server-setup/setup_controller.sh	VM4	Installs Python deps + iproute2 + iptables
scripts/network_config.sh	Each VM	Configures static IP from role argument
scripts/run_baseline_test.sh	VM4	Runs no-mitigation experiment, outputs JSON results
scripts/run_mitigated_test.sh	VM4	Runs full mitigation experiment, outputs JSON results

VirtShield — Prevention of Distributed Denial of Service Attacks with Virtualization

Oladeji Ibrahim Olaleye · U22CYS1103 · AFIT Kaduna · 2026

Powered by Ainga Lab